# Linux Troubleshooting — Scenario #5: "Permission Denied" After a Deploy

DevOps on-call runbook • Syed Aftab • Ubuntu (WSL) • systemd + journalctl + file permissions • Reference for real incidents & interviews

**The alert (9:10 AM):** the orders API has been down since this morning's deploy. The pipeline said "SUCCESS". The service keeps failing.
**Root cause found:** the deploy script (running as root) deleted and recreated `/var/log/orders-api` as `root:root 750`. It gave the **file** `app.log` to `orders` but forgot the **folder**, so the `orders` service user couldn't enter it. **Fix:** `chown orders:orders /var/log/orders-api`.

Why this happens so often: deploy scripts run as **root**, and root ignores permissions, so the deploy never notices a problem. The app runs as its **own unprivileged user**, and that user is the one that hits the wall.

### *The methodology*

| # | Phase | Commands | Goal |
|---|---|---|---|
| 1 | Confirm | `curl http://localhost:8081` | Is it really down? Connection refused = nothing listening |
| 2 | Locate | `systemctl status orders-api` | Crash loop? Exit code? First clue in the last log lines |
| 3 | Identify | `journalctl -u`, `ls -l`, `namei -l`, `sudo -u` | Full error, impact, which level of the path blocks it, prove it as the app user |
| 4 | Fix | `chown orders:orders` on the **folder** | Least-privilege fix — no chmod 777 |
| 5 | Verify | `status`, `curl`, `namei -l` | Running, answering, path open for the app only |
| 6 | Prevent | `LogsDirectory=`, `install -d`, health check | Deploys can't break it again, and can't lie about success |

## SETUP  Building the lab (healthy baseline first)

**0a. A dedicated service user** — real apps never run as root or as a person

```
sudo useradd --system --no-create-home --shell /usr/sbin/nologin orders
id orders
uid=997(orders) gid=986(orders) groups=986(orders)
```

| Part | Meaning |
|---|---|
| `useradd` | create a user (low-level tool; `adduser` is the friendlier Debian wrapper) |
| `--system` | system account: UID below 1000, like `www-data` |
| `--no-create-home` | no /home/orders folder — a service doesn't need one |
| `--shell /usr/sbin/nologin` | nobody can log in as this user; it only exists to run the app |
| `id USER` | show UID, main group (GID) and other groups. `id -u` = UID only |
| why a separate user? | if the app is hacked, the attacker gets only `orders`' rights, not root's |

**root vs. orders:** root (UID 0) **ignores permissions** and never sees "Permission denied". `orders` (UID 997) is limited by them.

**0b. Folders and the app**

```
sudo mkdir -p /opt/orders-api /var/log/orders-api
sudo chown orders:orders /var/log/orders-api
sudo tee /opt/orders-api/app.py > /dev/null <<'EOF'   ... (Python web app) ...
ls -ld /var/log/orders-api     # drwxr-xr-x 2 orders orders ... /var/log/orders-api
ls -l /opt/orders-api          # -rw-r--r-- 1 root root 1033 ... app.py
```

| Part | Meaning |
|---|---|
| `mkdir -p` | create folders; `-p` = make parents too, no error if they exist |
| `/opt/orders-api` | app code. `/opt` = optional / add-on software |
| `/var/log/orders-api` | the app's logs — the standard place for logs |
| `chown orders:orders` | **ch**ange **own**er to user orders, group orders |
| `ls -l` vs `ls -ld` | `-l` lists what's **inside** a folder; `-d` shows the **folder itself** |
| `app.py` root-owned, `rw-r--r--` | the app can read and run its code but **can't change it** — good practice |

The key line of the app: `logging.basicConfig(filename="/var/log/orders-api/app.log", ...)` — it opens the log file **at startup**. If it can't, the app crashes.

### *How to read a permission string*

```
drwxr-xr-x  2  orders  orders  4096  Oct 29 07:12  /var/log/orders-api
|\_/\_/\_/     owner   group
| u  g  o
```

| Part | Meaning |
|---|---|
| 1st character | d = directory, `-` = file, `l` = link |
| chars 2–4 | **owner** (user) permissions |
| chars 5–7 | **group** permissions |
| chars 8–10 | **others** (everyone else) |

| Letter | On a FILE | On a FOLDER |
|---|---|---|
| `r` (4) | read its contents | **list** the names inside (ls) |
| `w` (2) | change it | **create, delete, rename** files inside |
| `x` (1) | run it as a program | **enter / pass through** it to reach anything inside |

**0c. The systemd service**

```
[Service]
User=orders
Group=orders
ExecStart=/usr/bin/python3 /opt/orders-api/app.py
Restart=on-failure
RestartSec=5
```

| Line | Meaning |
|---|---|
| `User=orders` / `Group=orders` | **run the app as orders, not root**. Without User=, systemd runs services as root |
| `ExecStart=` | the command; full paths required |
| `Restart=on-failure` | restart only after a crash (non-zero exit) — gentler than Scenario #4's always |
| `RestartSec=5` | wait 5 s between attempts |

**0d. Start it and record the healthy baseline**

```
sudo systemctl daemon-reload
sudo systemctl enable --now orders-api
curl http://localhost:8081                # orders-api OK
sudo cat /var/log/orders-api/app.log      # INFO:root:orders-api starting on port 8081
ps -o user,pid,cmd -C python3
  orders  325324 /usr/bin/python3 /opt/orders-api/app.py
```

| Part | Meaning |
|---|---|
| `curl URL` (no -I) | show the response **body** |
| `ps -o user,pid,cmd` | choose columns: user, PID, command |
| `-C python3` | only processes whose **C**ommand name is python3 |
| baseline | curl OK + log lines growing + process user `orders` = what "healthy" looks like |

**0e. The bad deploy** (ran as root; explained line by line in Step 6)

```
sudo bash /opt/orders-api/deploy.sh
Deploy complete: SUCCESS
```

## STEP 1 Confirm — is it really down?

```
curl http://localhost:8081
curl: (7) Failed to connect to localhost port 8081 after 1 ms: Could not connect to server
```

Error 7 = **nothing is listening** on port 8081 — the app isn't running at all (not a 500 from a running app). The deploy's SUCCESS was wrong.

## STEP 2 Locate — systemctl status

```
systemctl status orders-api
 Active: activating (auto-restart) (Result: exit-code) since ... 07:21:06; 4s ago
Process: 326293 ExecStart=/usr/bin/python3 /opt/orders-api/app.py (code=exited, status=1/FAILURE)
... python3[326367]: PermissionError: [Errno 13] Permission denied: '/var/log/orders-api/ap>
```

| Clue | Meaning |
|---|---|
| `activating (auto-restart)` | it crashed and systemd is waiting RestartSec (5 s) to try again → a **crash loop** |
| `status=1/FAILURE` | the program exited with code 1 = an error |
| PID 326293 vs log PID 326367 | different PIDs = already restarted; every attempt is a new process |
| `PermissionError: [Errno 13]` | **the clue**. Errno 13 = EACCES: the **operating system** refused access, not Python |
| `ap>` | path cut off by the pager — get the full text from journalctl |

**Active states:** `active (running)` = up • `failed` = crashed, systemd gave up • `activating (auto-restart)` = crashed, about to retry.

## STEP 3 Identify — full error, impact, and the exact blocked level

**3a. Full traceback with journalctl**

```
journalctl -u orders-api -n 30 --no-pager
... filename="/var/log/orders-api/app.log",
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
... File "/usr/lib/python3.14/logging/__init__.py", line 1248, in _open
... PermissionError: [Errno 13] Permission denied: '/var/log/orders-api/app.log'
... orders-api.service: Main process exited, code=exited, status=1/FAILURE
```

| Part | Meaning |
|---|---|
| `-u orders-api` | only this unit's logs |
| `-n 30` | last 30 lines |
| `--no-pager` | print straight to the terminal: long lines **wrap** instead of being cut off, and output can be piped |
| `^^^^^` | Python underlining the exact code that failed: the `filename=` line in `logging.basicConfig` |

**Reading a Python traceback:** it lists the chain of calls **oldest first**, and the **last line is the actual error**. Read it bottom-up: the last line says *what* went wrong, the lines above say *where*.

**3b. Impact — how many restarts?**

```
journalctl -u orders-api --since "07:20" --no-pager | grep -c "Scheduled restart"
55
```

| Part | Meaning |
|---|---|
| `--since "07:20"` | from just before the deploy |
| `\| grep -c` | `-c` = **c**ount matching lines instead of printing them |
| `"Scheduled restart"` | the line systemd logs on every restart |
| 55 × ~5 s | ≈ 4.5 minutes down. A crash loop also floods the journal with the same traceback |

**Ticket impact line:** orders-api down since ~07:21 (deploy). Crash-looping, 55 restarts, PermissionError on its log file.

**3c. The file — looks perfect (the trap)**

```
sudo ls -l /var/log/orders-api/app.log
-rw-r----- 1 orders orders 0 Oct 29 07:20 /var/log/orders-api/app.log
```

| Clue | Meaning |
|---|---|
| owner `orders`, `rw-` | the app's user owns it and can write it — so why denied? |
| size `0`, time 07:20 | an empty file created during the deploy — the old log lines were **deleted** |
| needed `sudo` to look | a hint: your own user couldn't see it either |

**The rule:** to open a file, Linux first checks **every folder in the path**. You need **x (enter)** on `/`, `/var`, `/var/log` and `/var/log/orders-api`. If any one blocks you, the file's own permissions are never checked.

**3d. Every level of the path — namei**

```
namei -l /var/log/orders-api/app.log
f: /var/log/orders-api/app.log
drwxr-xr-x root root   /
drwxr-xr-x root root   var
drwxrwxr-x root syslog log
drwxr-x--- root root   orders-api
                       app.log - Permission denied
```

| Part | Meaning |
|---|---|
| `namei` | follow a path **name** level by level |
| `-l` | long format: permissions, owner, group of every level (like ls -l) |

| Level | Perms | Owner:group | orders is... | Pass? |
|---|---|---|---|---|
| `/` | `drwxr-xr-x` | root:root | other → r-x | Yes |
| `var` | `drwxr-xr-x` | root:root | other → r-x | Yes |
| `log` | `drwxrwxr-x` | root:syslog | other → r-x | Yes |
| `orders-api` | `drwxr-x---` | **root:root** | **other → ---** | **NO** |
| `app.log` | `-rw-r-----` | orders:orders | owner → rw- | never reached |

The folder belongs to **root:root** (baseline: orders:orders). `orders` isn't root and isn't in group root, so it's "others" → `---` → no x → can't enter. Like owning a locker inside a room you can't get into.

**3e. Prove it as the app's user — before changing anything**

```
sudo -u orders ls /var/log/orders-api
ls: cannot open directory '/var/log/orders-api': Permission denied
sudo -u orders touch /var/log/orders-api/app.log
touch: setting times of '/var/log/orders-api/app.log': Permission denied
```

| Part | Meaning |
|---|---|
| `sudo -u orders` | run the command **as user orders** (-u = user). Works even with nologin — it runs one command, it doesn't log in |
| `sudo CMD` (no -u) | runs as root — useless for this test, root ignores permissions |
| `ls DIR` | needs r + x on the folder |
| `touch FILE` | update the file's timestamp — needs write access, so it tests what the app does |
| other services | `sudo -u www-data ...` (nginx), `sudo -u postgres ...` (database) |

## STEP 4 Fix — the right way, not the fast way

| Fix | Command | Result |
|---|---|---|
| Panic fix | `chmod -R 777 DIR` | works, but **every user** can read/change/delete the logs (often customer data, tokens). A security incident of its own |
| Half fix | `chmod o+x DIR` | works, but lets everyone in, root still owns it, next deploy breaks it again |
| **Right fix** | `chown orders:orders DIR` | only orders (and root) can use the folder — least privilege |

```
sudo chown orders:orders /var/log/orders-api
sudo chmod 750 /var/log/orders-api          # safety step: already 750, changed nothing
sudo -u orders touch /var/log/orders-api/app.log    # no output = works
```

**Numeric permissions** — r = 4, w = 2, x = 1, one digit each for owner / group / others:

| Digit | For | Sum | Letters | Meaning |
|---|---|---|---|---|
| 7 | owner (orders) | 4+2+1 | rwx | full access |
| 5 | group (orders) | 4+0+1 | r-x | list and enter, no write |
| 0 | others | 0 | --- | nothing — logs stay private |

**Why chown alone was enough:** owner and permissions are **separate**. `chown` changes **who** owns it; `chmod` changes the **letters**. Neither touches the other. The folder was already `rwxr-x---`. chown didn't move or "inherit" any permissions — it moved `orders` from the **others** seat (`---`) to the **owner** seat (`rwx`). Same permissions, different seat.

| Question | Answer |
|---|---|
| Does root still own it? | No — orders owns the folder now. But **root can still access everything**: root ignores permissions. Ownership decides owner vs. others for everyone else |
| Does chown on a folder change the files inside? | No. Only the folder. `chown -R` / `chmod -R` = **R**ecursive: folder + everything inside. Not needed here: app.log was already orders:orders |
| Is anything inherited from the parent folder? | No, not automatically (except special settings like setgid or ACLs) |

## STEP 5 Verify

```
systemctl status orders-api
 Active: active (running) since Thu 2026-10-29 07:32:34 EDT
 ... Scheduled restart job ... / Started orders-api.service
curl http://localhost:8081        # orders-api OK
namei -l /var/log/orders-api/app.log
drwxr-x--- orders orders orders-api
                        app.log - Permission denied   (as YOU = correct)
```

| Check | Meaning |
|---|---|
| running since 07:32:34 | **recovered on its own**: the crash loop's next retry after the fix succeeded. Restart=on-failure helped this time — no restart needed |
| `curl` OK | answering requests again |
| `orders orders` on orders-api | owned by the app again |
| still denied as **you** | correct — others get 0, logs are private. `sudo namei -l` shows the whole path |

**Ticket summary:** Deploy recreated /var/log/orders-api as root:root 750, so the orders service user couldn't enter it. Fixed with chown orders:orders on the folder. Service recovered automatically at 07:32:34. Old logs were deleted by the deploy.

## STEP 6 The bad deploy explained — and how to prevent it

```
umask 027
rm -rf /var/log/orders-api
mkdir -p /var/log/orders-api
touch /var/log/orders-api/app.log
chown orders:orders /var/log/orders-api/app.log
systemctl restart orders-api
echo "Deploy complete: SUCCESS"
```

| Line | What it did | Problem |
|---|---|---|
| `umask 027` | new items lose group write and all "others" bits: folders 750, files 640 | fine on its own |
| `rm -rf DIR` | deleted the whole log folder | **all old logs lost** (app.log was 0 bytes). Logs are evidence |
| `mkdir -p DIR` | recreated it **as root** → root:root 750 | orders became "others" with 0 |
| `touch` + `chown ... app.log` | made the file and gave it to orders | **fixed the file, forgot the folder** |
| `systemctl restart` | restarted the app | crashed instantly |
| `echo "SUCCESS"` | printed success | **checked nothing** — the pipeline lied |

**1. Let systemd own the log folder (best fix)** — add to [Service]:

```
LogsDirectory=orders-api
```

On every start, systemd creates /var/log/orders-api if missing and gives it to the unit's User=. A deploy can no longer leave the wrong owner.

**2. If a script must create folders, set owner and mode at creation:**

```
install -d -o orders -g orders -m 750 /var/log/orders-api
```

| Part | Meaning |
|---|---|
| `install -d` | create a **d**irectory |
| `-o orders -g orders` | owner and group, set immediately |
| `-m 750` | mode (permissions). One command = mkdir + chown + chmod, nothing forgotten |

**3. Never rm -rf logs in a deploy** — rotate or archive them (`logrotate`).

**4. Make the deploy prove it worked** — replace the blind echo:

```
sleep 3
curl -fsS http://localhost:8081 || { echo "Deploy FAILED: health check"; exit 1; }
echo "Deploy complete: SUCCESS"
```

| Part | Meaning |
|---|---|
| `curl -f` | **f**ail with a non-zero exit code on HTTP errors (without it, curl "succeeds" on a 500) |
| `-sS` | silent, but still **S**how errors |
| `\|\| { ...; exit 1; }` | if curl failed: print a message and exit 1 → the pipeline turns red |

**5. After any permission change, test as the service user:** `sudo -u orders ...`

### *Lab cleanup*

```
sudo systemctl disable --now orders-api
sudo rm /etc/systemd/system/orders-api.service
sudo systemctl daemon-reload
sudo rm -rf /opt/orders-api /var/log/orders-api
sudo userdel orders
```

| Part | Meaning |
|---|---|
| `disable --now` | stop now and don't start at boot |
| `rm` unit + `daemon-reload` | remove the service and make systemd forget it |

| Part | Meaning |
|---|---|
| `rm -rf` | remove the lab folders (fine here — it's a lab, not production logs) |
| `userdel orders` | delete the service user |

## REF Full command reference

| Command | What | Why here |
|---|---|---|
| `curl URL` | request a page (body) | confirm down / back up |
| `systemctl status NAME` | state + last logs | found the crash loop + PermissionError |
| `journalctl -u NAME -n 30 --no-pager` | full recent logs, no truncation | full traceback and path |
| `journalctl -u NAME --since "T" \| grep -c "TEXT"` | count events | impact: 55 restarts |
| `ls -l FILE` / `ls -ld DIR` | permissions of a file / of the folder itself | the file looked fine |
| `namei -l PATH` | permissions of every level of a path | found the blocked folder |
| `sudo -u USER CMD` | run as another user | prove it as the app's user |
| `chown USER:GROUP PATH` | change owner | **the fix** |
| `chmod 750 PATH` | set permissions by number | make the mode explicit |
| `chown -R` / `chmod -R` | recursive: include everything inside | when files inside are wrong too |
| `id USER` / `id -u` | UID, groups / UID only | check the service user |
| `ps -o user,pid,cmd -C NAME` | which user a process runs as | baseline: runs as orders |
| `useradd --system ... USER` | create a service account | lab setup |
| `install -d -o U -g G -m 750 DIR` | create folder with owner + mode | prevention |
| `LogsDirectory=NAME` | systemd creates and owns the log folder | prevention |
| `curl -fsS URL \|\| exit 1` | health check that fails the deploy | prevention |

### *Interview answer (say this)*

*"After a deploy the service was crash-looping. systemctl status showed activating (auto-restart) and a PermissionError, so I read the full traceback with journalctl -u and counted restarts to report impact. The log file itself was owned by the service user, which is the trap — Linux needs execute permission on every directory in the path. namei -l showed the deploy had recreated the log directory as root:root 750, so the service user was 'other' and couldn't enter it. I proved it with sudo -u as the service user, fixed it with chown on the directory — not chmod 777 — and the service recovered on its next retry. To prevent it: systemd LogsDirectory= or install -d with owner and mode, never delete logs in a deploy, and a curl -f health check so the pipeline can't report success on a broken release."*